# Where the FFT Sphere Solver Runs

## Memory, precision and hardware for the Foldy–Lax sphere, measured on Graviton CPUs and an NVIDIA A10G

31 October 2026


**Abstract**

The FFT-accelerated Foldy–Lax solver for a voxelised sphere (`compute_sphere_foldy_lax_fft_gpu`) was measured from $n_{\mathrm{sub}} = 12$ to 96 cells across the diameter, up to 463 400 cells and 4.2 million unknowns, on eight Graviton3 cores and on an NVIDIA A10G, at $k_S a = 1$. Four results follow.

*Memory.* The GMRES solver allocated its whole Krylov basis, (maxiter + 1) vectors, before the first step: 13.4 GB of a 27.7 GB peak at $n_{\mathrm{sub}} = 96$, for solves that took four steps. It now grows the basis a vector a step, with results bit-identical to before. What remains is dominated by the padded grids of the FFT product.

*Speed.* At $n_{\mathrm{sub}} = 96$ the A10G forms the FFT product 15 times faster than the eight cores in complex128, and about 50 times faster in complex64. The solve then stops being the cost: the kernel and the right-hand sides, built on the CPU, take most of the time.

*Precision.* With the kernel built in complex128 and only the solve in complex64, the far field agrees with the complex128 solution to $3.9 \times 10^{-9}$ at a GMRES tolerance of $10^{-6}$, against a discretisation error of $6.7 \times 10^{-5}$. Single precision costs nothing measurable.

*Design.* The two halves want different machines: the kernel build, in complex128, parallel and mostly independent of frequency, on many CPU cores; the solve, in complex64 and limited by memory bandwidth, on a GPU. The canonical kernel that passes between them is 160 MB at $n_{\mathrm{sub}} = 96$ in complex64.


## Contents


**1 The solver and the measurement** **2**

1.1 What is solved . . . 2

1.2 The test case . . . 2

1.3 The probe . . . 2

1.4 The machines . . . 3

**2 Memory** **3**

2.1 The Krylov basis was allocated in full . . . 3

2.2 What remains . . . 3

**3 Time on the CPU and on the GPU** **4**

**4 Precision** **4**

**5 Where the time goes** **5**

**6 A kernel stage and a solve stage** **6**

**7 Limits of these measurements** **7**

**8 Reproduction** **7**

# 1 The solver and the measurement

## 1.1 What is solved

A sphere of radius $a = 0.5\,\mathrm{m}$ is voxelised on a cubic grid of $n_{\mathrm{sub}}$ cells across its diameter; the cells whose centres lie inside it, $N$ of them, each carry nine unknowns (displacement and Voigt strain). The Foldy–Lax system couples every pair of cells through the cell-averaged propagator. Its product with a vector is a convolution, formed by FFT on a zero-padded grid of $n_P^3$ points per component, with $n_P$ the smallest odd length at least $2n_{\mathrm{sub}} - 1$ whose only prime factors are 3, 5 and 7 (`fast_odd_size`): $n_P = 225$ at $n_{\mathrm{sub}} = 96$. The kernel is stored only at the canonical wavenumbers of the cube group, about $1/46$ of the grid (§6). GMRES (`torch_gmres`) solves one column for each orbit of the right-hand sides under the symmetries: three Taylor patterns, which give the composite $9 \times 9$ T-matrix, and six plane-wave columns, which give the far field. The other columns follow by symmetry.

The kernel is always built on the CPU in complex128: the cell-averaged pair blocks come from a wavenumber series whose recursions need the precision (`cell_average_kseries.py`). Only the solve runs in the device's dtype, on the kernel rounded to it.

## 1.2 The test case

The background is the project's test medium, $\alpha = 5000\,\mathrm{m/s}$, $\beta = 3000\,\mathrm{m/s}$, $\rho = 2500\,\mathrm{kg/m^3}$; the sphere's contrast is $\Delta\lambda = 2\,\mathrm{GPa}$, $\Delta\mu = 1\,\mathrm{GPa}$, $\Delta\rho = 100\,\mathrm{kg/m^3}$; the frequency is fixed at $k_S a = 1$, so that refining $n_{\mathrm{sub}}$ refines the discretisation of the same physical problem. GMRES converged in three or four steps at every size.

## 1.3 The probe

`scripts/probe_fft_sphere_scaling.py` runs each size in a fresh process, so that its peak memory is its own, and records the time to build the kernel, each GMRES solve (iterations, products, time), the total, the peak resident memory of the process, and on CUDA the allocator's peak. The k-series store, the universal coefficients of the pair blocks (`numerics.yml`: `kseries_store`), grows on first use by the offsets a size needs; every run reported here found it already grown.

`scripts/measure_fft_sphere_precision.py` compares precisions and the exact Mie solution (§4).

### 1.4 The machines

**Table 1.** The two AWS instances (Sydney, `ap-southeast-2`), each used through the project's image (`aws/README.md`): Python 3.12, NumPy 2.4.2, torch 2.10.0 (the CUDA 12.8 build on the GPU).

| Instance | CPU | Memory | GPU |
|---|---|---|---|
| r7g.2xlarge | 8 Graviton3 cores (arm64) | 64 GiB | — |
| g5.xlarge | 4 vCPUs: 2 AMD EPYC cores, hyperthreaded | 16 GiB | NVIDIA A10G, 24 GB |

The g5.xlarge was chosen for its GPU; its two physical cores matter for the stages that stay on the CPU (§5). Two GPU runs that shared the instance with another job are excluded from the data; the GPU rows here each had the instance to themselves.

## 2 Memory

### 2.1 The Krylov basis was allocated in full

`torch_gmres` has no restart, and it allocated the basis for every iteration it might take, (maxiter + 1) × $9N$ complex numbers, before the first. At the default maxiter = 200 that is 13.4 GB at $n_{\text{sub}} = 96$ in complex128, for solves that converged in four steps. Lowering maxiter to 20 removed almost exactly that much (table 2).

**Table 2.** Peak resident memory on r7g.2xlarge, complex128, with the basis allocated in full. The drop is the basis's: 3.44 against 3.56 GB at $n_{\text{sub}} = 64$, 11.16 against 12.01 GB at $n_{\text{sub}} = 96$.

| | | maxiter = 200 | | maxiter = 20 | |
|---|---|---|---|---|---|
| $n_{\text{sub}}$ | cells | basis (GB) | peak (GB) | basis (GB) | peak (GB) |
| 64 | 137 376 | 3.98 | 8.88 | 0.42 | 5.44 |
| 96 | 463 400 | 13.41 | 27.71 | 1.40 | 16.55 |

The basis is now a list to which Arnoldi appends each vector it makes, so its memory follows the iterations taken, ($n_{\text{iter}} + 1$) × $9N$, and maxiter can be set generously against hard cases at no cost; only the (maxiter + 1) × maxiter Hessenberg matrix is sized by it. The arithmetic is unchanged: against the previous version, solutions, iteration counts and residuals are bit-identical in both precisions, for converging solves, solves that reach maxiter, and the breakdown branch. With $10^6$ unknowns, maxiter = 200 and four steps, the peak fell from 3.29 to 0.24 GB. Every GPU measurement below uses the new version.

### 2.2 What remains

Without the full basis, the A10G's peak at $n_{\text{sub}} = 96$ in complex128 is 14.6 GB, of which the basis is 0.33 GB and the canonical kernel 0.32 GB. One padded grid stack, nine components of

$n_P^3$ complex numbers, is 1.64 GB, so the remainder is about eight to nine stacks: the temporaries of the FFT product (the packed grid, its transform, the product with the kernel and its inverse transform), as inferred from their size; it has not been profiled. Two consequences follow. Device memory scales as $n_P^3$, not as $N$, and the padding to a 3-5-7-smooth length makes $n_P^3$ at $n_{\text{sub}} = 96$ $(225/191)^3 = 1.63$ times the minimum. Reducing the temporaries, by transforming in place or component by component, is the next memory saving.

The host's peak also includes about 2.5 to 3 GB that does not grow with $N$: 2.7 to 3.1 GB at $n_{\text{sub}} = 24$, where the solver's own arrays are well under 1 GB. It is absent from the runs made while the k-series store held only the offsets of small sizes (0.36 GB at $n_{\text{sub}} = 12$), so the memory-mapped store is the likely part of it.

## 3 Time on the CPU and on the GPU

**Table 3.** Time per FFT product, and the total for one sphere (both the composite T-matrix and the far-field columns), in seconds. CPU: r7g.2xlarge, complex128, gmres_tol $10^{-8}$. GPU: g5.xlarge, complex128 at $10^{-8}$ (four steps a solve) and complex64 at $10^{-6}$ (three).

| | | seconds per product | | | total seconds | | |
|---|---|---|---|---|---|---|---|
| $n_{\text{sub}}$ | cells | CPU | GPU c128 | GPU c64 | CPU | GPU c128 | GPU c64 |
| 24 | 7 208 | 0.0344 | 0.0184 | 0.0118 | 3.0 | 2.6 | 2.0 |
| 32 | 17 256 | 0.0819 | 0.0119 | 0.0074 | 6.0 | 2.7 | 2.4 |
| 48 | 57 856 | 0.437 | 0.0369 | 0.0138 | 26.9 | 6.6 | 5.2 |
| 64 | 137 376 | 1.021 | 0.0694 | 0.0237 | 60.8 | 13.2 | 10.5 |
| 96 | 463 400 | 4.911 | 0.318 | 0.096 | 283.1 | 50.2 | 37.3 |

At $n_{\text{sub}} = 96$ the A10G forms a product 15 times faster than the eight Graviton cores in complex128 (0.318 against 4.6 to 4.9 s), and 48 to 51 times faster in complex64 (0.096 s). Complex128 is the A10G's weak mode for arithmetic, yet it still wins by an order of magnitude: the product is a sequence of FFTs and pointwise multiplications, limited by memory bandwidth more than by floating-point rate. Halving the bytes, complex64 makes the product 3.3 times faster still, and the tolerance it can reach, $10^{-6}$, takes three steps instead of four (42 products a sphere against 53 or 54).

The totals tell a different story from the products. In complex64 at $n_{\text{sub}} = 96$ the solves take 4.0 s of 37.3: the rest is on the CPU. On the r7g.2xlarge, whose eight cores ran close to 800% busy, the kernel took 11.7 s; on the g5.xlarge's two cores it takes 18.5 s; and about 15 s more goes to stages that the probe did not separate. Section 5 finds them.

## 4 Precision

Each sphere was solved three times on the A10G with the same complex128 kernel: in complex128 at gmres_tol $10^{-10}$, the reference, and in complex64 at $10^{-5}$ and at $10^{-6}$ (single precision cannot reach the reference's tolerance). Two errors are compared, both relative, in the maximum over the components and angles: the *precision* error, complex64 against complex128, on the composite

T-matrix and on the far field; and the *discretisation* error, complex128 against the exact elastic Mie solution, on the far field. The far field is for a $P$ wave incident along $z$, observed at nine angles in the $z$–$x$ plane at $5 \times 10^4$ radii, the distance at which `gate_sphere_cell_average_vs_mie.py` established that the asymptotic and exact fields agree. The discretisation error is given raw and with the voxel volume scaled to the sphere's, which removes the staircase's dilution but not its shape.

**Table 4.** The error of solving in complex64, beside the error of the discretisation (g5.xlarge, $k_S a = 1$).

| | | c128 against Mie | | | c64 against c128 | |
|---|---|---|---|---|---|---|
| $n_{\text{sub}}$ | cells | raw | volume-corrected | gmres_tol | T-matrix | far field |
| 32 | 17 256 | $4.97 \times 10^{-3}$ | $8.25 \times 10^{-4}$ | $10^{-5}$ | $4.8 \times 10^{-7}$ | $8.4 \times 10^{-7}$ |
| | | | | $10^{-6}$ | $6.4 \times 10^{-8}$ | $4.9 \times 10^{-9}$ |
| 64 | 137 376 | $7.14 \times 10^{-4}$ | $1.47 \times 10^{-4}$ | $10^{-5}$ | $5.1 \times 10^{-7}$ | $9.0 \times 10^{-7}$ |
| | | | | $10^{-6}$ | $8.1 \times 10^{-8}$ | $3.8 \times 10^{-9}$ |
| 96 | 463 400 | $2.69 \times 10^{-4}$ | $6.68 \times 10^{-5}$ | $10^{-5}$ | $5.1 \times 10^{-7}$ | $9.2 \times 10^{-7}$ |
| | | | | $10^{-6}$ | $8.6 \times 10^{-8}$ | $3.9 \times 10^{-9}$ |

The precision error is set by the GMRES tolerance, not by the word length. Tightening it from $10^{-5}$ to $10^{-6}$ lowers the far-field difference from about $9 \times 10^{-7}$ to $4 \times 10^{-9}$, below single precision's unit roundoff, since the far field sums over all the cells; and it costs nothing measurable (31.3 against 30.8 s at $n_{\text{sub}} = 96$). At $10^{-6}$ the precision error is about 17 000 times smaller than the discretisation error at $n_{\text{sub}} = 96$, and the complex64 and complex128 far fields agree with Mie to the three figures shown at every size.

The discretisation error falls at about second order in $n_{\text{sub}}$ (the volume-corrected column: a factor 5.6 from 32 to 64 and 2.2 from 64 to 96, against 4 and 2.25 for second order), so a gmres_tol of $10^{-5}$ would begin to matter only at sizes far beyond these; $10^{-6}$ is the safe default. Against a target of $10^{-5}$ relative to the exact solution, the limit at these sizes is the staircase, not the solver.

## 5 Where the time goes

A profile of one solve at $n_{\text{sub}} = 96$ in complex128 on the g5.xlarge (`scripts/data/profile_fft_sphere_gpu_g5.xlarge_complex128_n96.txt`) accounts for the stages the probe did not separate (table 5). GPU work is asynchronous, so the solves' time appears where the solver waits for the device, inside `torch_gmres`.

The incident patterns were built cell by cell, an identity block and six small strain tensors for each of 463 400 cells, though every entry is one product: 1 or $\frac{1}{2}$ times a component of the cell's offset from the centre. Built for all cells at once, they take 0.45 s against 5.55 s for the loop, on a two-core x86-64 machine (11.7 s on the g5.xlarge). The composite T-matrix, $\sum_n t_n \psi_n$ over the cells, is now formed in batches of 4096 cells that stay in cache: 0.35 against 1.16 s. Both are bit-identical to the loops, signed zeros included: NumPy sums along the first axis in order, and the running sum enters each batch as its first row (the cheaper $t \sum_n \psi_n$ differs at $10^{-15}$ and was not used). The four solvers that built the T-matrix by the loop all use the new form, and their outputs, uniform and graded, in both precisions, are unchanged bit for bit.

**Table 5.** One solve, $n_{\text{sub}} = 96$, complex128, g5.xlarge (52.3 s in the solver), before the vectorisation below.

| Stage | seconds | where |
|---|---|---|
| kernel build (`_point_canonical_kernel`) | 19.3 | CPU |
| cell-averaged pair blocks (k-series) | 7.1 | depends on the frequency |
| symmetry and index tables | about 11 | depends on $n_{\text{sub}}$ only |
| GMRES solves | 16.9 | GPU |
| incident patterns (`_build_incident_field_coupled`) | 11.7 | CPU, a loop over cells |
| copies from the device | 3.1 | |
| the solver's own lines, the composite T-matrix among them | 2.2 | CPU |
| symmetry applied to the solved columns | 1.2 | CPU |

With these, a complex64 solve at $n_{\text{sub}} = 96$ on the g5.xlarge should take roughly 26 s instead of 37 (not yet measured there), of which the kernel build is 18.5. That is the stage the design below moves off the GPU instance.

## 6 A kernel stage and a solve stage

The measurements separate the solver into two stages with different needs.

- The *kernel stage* runs in complex128, on the CPU, and parallelises well (the eight Graviton cores ran close to 800% busy). More than half of it, at $n_{\text{sub}} = 96$, is symmetry and index tables that depend only on $n_{\text{sub}}$; the incident patterns depend only on the cell centres. Only the pair blocks, 7.1 s here, depend on the frequency and the medium.
- The *solve stage* runs in complex64, on the GPU, limited by memory bandwidth, and needs few CPU cores.

What passes between them is the canonical kernel, $K \times 9 \times 9$ complex numbers at the $K$ canonical wavenumbers (table 6), built in complex128 and rounded to complex64 for the solve.

**Table 6.** The canonical kernel handed from the kernel stage to the solve stage.

| $n_{\text{sub}}$ | $n_P$ | canonical wavenumbers $K$ | complex128 | complex64 |
|---|---|---|---|---|
| 64 | 135 | 54 740 | 71 MB | 35 MB |
| 96 | 225 | 246 905 | 320 MB | 160 MB |
| 128 | 315 | 669 920 | 868 MB | 434 MB |

So the stages can run on different machines, with the kernel passed through object storage in seconds: the kernel stage on many Graviton cores, as a batch array over frequencies on Spot instances, with the $n_{\text{sub}}$-only tables computed once and kept; the solve stage on a small GPU instance, starting as each kernel is ready. In complex64 the A10G's 24 GB holds $n_{\text{sub}} = 128$ (19.6 GB by extrapolation of the measured sizes); $n_{\text{sub}} = 160$ would need about 40 GB, a 48 GB GPU or fewer FFT temporaries (§2). Building this needs the solver split into a function that builds and saves the canonical kernel and one that solves from it, with the present function their composition; it has not been done.

## 7 Limits of these measurements

- One frequency ($k_S a = 1$) and one moderate contrast, at which GMRES takes three or four steps. Stronger contrast or higher frequency takes more, and the Krylov basis, now sized by the steps taken, grows with them.
- The sphere is voxelised as a staircase; the discretisation error of table 4 is the staircase's, and a body the lattice resolves smoothly would set a lower bar for the precision error to clear.
- Each configuration was run once. Where two runs measured the same stage, they agreed to within about 6% (the product at $n_{\text{sub}} = 96$ on the CPU, 4.91 and 4.61 s; the GPU solves, 16.97 and 16.85 s), but no spread is reported.
- The memory attributed to FFT temporaries and to the k-series store is inferred from sizes, not profiled.
- The timings of the vectorised stages (§5) were measured on a two-core x86-64 machine, not on the instances.

## 8 Reproduction

The data are in `scripts/data/`: `fft_sphere_scaling_r7g.2xlarge_cpu_complex128.jsonl`, `fft_sphere_scaling_g5.xlarge_cuda_complex128.jsonl`, `fft_sphere_scaling_g5.xlarge_cuda_complex64.jsonl`, `fft_sphere_precision_g5.xlarge_cuda.jsonl` and the profile above. The commits are 8818dab (the growing Krylov basis), 4af0b78 (the data) and 9d72ca1 (the vectorised incident patterns and composite T-matrix). With the AWS tooling of `aws/README.md`, from a checkout:

```
python3 aws/msc_aws.py run --on ec2 -- python scripts/probe_fft_sphere_scaling.py \
    --device cpu --n-sub 24,32,48,64,96 --warm-store
python3 aws/msc_aws.py run --on ec2 --gpu -- python scripts/probe_fft_sphere_scaling.py \
    --device cuda --dtype complex128 --n-sub 24,32,48,64,96 --warm-store
python3 aws/msc_aws.py run --on ec2 --gpu -- python scripts/probe_fft_sphere_scaling.py \
    --device cuda --dtype complex64 --tol 1e-6 --n-sub 24,32,48,64,96
python3 aws/msc_aws.py run --on ec2 --gpu -- python scripts/measure_fft_sphere_precision.py \
    --device cuda --n-sub 32,64,96
```

The same scripts run locally with `PYTHONPATH=.`; `--fit-only` reprints a table from its data file. `--warm-store` grows the k-series store before the measured sizes, which otherwise include its one-time growth; at $n_{\text{sub}} = 96$ that growth took roughly 45 minutes on the r7g.2xlarge.